

Table 1: Long-term forecasting results. All results are averaged from four different forecasting horizons: $H \in \{24, 36, 48, 60\}$ for ILI and $\{96, 192, 336, 720\}$ for the others. A lower value indicates better performance. **Red**: the best, **Blue**: the second best. Our full results are in Appendix D.

Methods	TIME-LLM (Ours)		GPT4TS (2023a)		DLinear (2023)		PatchTST (2023)		TimesNet (2023)		FEDformer (2022)		Autoformer (2021)		Stationary (2022)		ETSformer (2022)		LightTS (2022a)		Informer (2021)		Reformer (2020)	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
<i>ETTh1</i>	0.408	0.423	0.465	0.455	0.422	0.437	0.413	0.430	0.458	0.450	0.440	0.460	0.496	0.487	0.570	0.537	0.542	0.510	0.491	0.479	1.040	0.795	1.029	0.805
<i>ETTh2</i>	0.334	0.383	0.381	0.412	0.431	0.446	0.330	0.379	0.414	0.427	0.437	0.449	0.450	0.459	0.526	0.516	0.439	0.452	0.602	0.543	4.431	1.729	6.736	2.191
<i>ETTm1</i>	0.329	0.372	0.388	0.403	0.357	0.378	0.351	0.380	0.400	0.406	0.448	0.452	0.588	0.517	0.481	0.456	0.429	0.425	0.435	0.437	0.961	0.734	0.799	0.671
<i>ETTm2</i>	0.251	0.313	0.284	0.339	0.267	0.333	0.255	0.315	0.291	0.333	0.305	0.349	0.327	0.371	0.306	0.347	0.293	0.342	0.409	0.436	1.410	0.810	1.479	0.915
<i>Weather</i>	0.225	0.257	0.237	0.270	0.248	0.300	0.225	0.264	0.259	0.287	0.309	0.360	0.338	0.382	0.288	0.314	0.271	0.334	0.261	0.312	0.634	0.548	0.803	0.656
<i>ECL</i>	0.158	0.252	0.167	0.263	0.166	0.263	0.161	0.252	0.192	0.295	0.214	0.327	0.227	0.338	0.193	0.296	0.208	0.323	0.229	0.329	0.311	0.397	0.338	0.422
<i>Traffic</i>	0.388	0.264	0.414	0.294	0.433	0.295	0.390	0.263	0.620	0.336	0.610	0.376	0.628	0.379	0.624	0.340	0.621	0.396	0.622	0.392	0.764	0.416	0.741	0.422
<i>ILI</i>	1.435	0.801	1.925	0.903	2.169	1.041	1.443	0.797	2.139	0.931	2.847	1.144	3.006	1.161	2.077	0.914	2.497	1.004	7.382	2.003	5.137	1.544	4.724	1.445
1 st Count	7		0		0		5		0		0		0		0		0		0		0		0	

Table 2: Short-term time series forecasting results on M4. The forecasting horizons are in $[6, 48]$ and the three rows provided are weighted averaged from all datasets under different sampling intervals. A lower value indicates better performance. **Red**: the best, **Blue**: the second best. More results are in Appendix D.

Methods	TIME-LLM (Ours)	GPT4TS (2023a)	TimesNet (2023)	PatchTST (2023)	N-HiTS (2023b)	N-BEATS (2020)	ETSformer (2022)	LightTS (2022a)	DLinear (2023)	FEDformer (2022)	Stationary (2022)	Autoformer (2021)	Informer (2021)	Reformer (2020)
Average SMAPE	11.983	12.69	12.88	12.059	12.035	12.25	14.718	13.525	13.639	13.16	12.780	12.909	14.086	18.200
Average MASE	1.595	1.808	1.836	1.623	1.625	1.698	2.408	2.111	2.095	1.775	1.756	1.771	2.718	4.223
Average OWA	0.859	0.94	0.955	0.869	0.869	0.896	1.172	1.051	1.051	0.949	0.930	0.939	1.230	1.775

are twice as prediction horizons. The evaluation metrics are symmetric mean absolute percentage error (SMAPE), mean absolute scaled error (MASE), and overall weighted average (OWA).

Results. Our brief results with unified seeds across all methods are in Tab. 2. TIME-LLM consistently surpasses all baselines, outperforming GPT4TS by **8.7%**. TIME-LLM remains competitive even when compared with the SOTA model, N-HiTS (Challu et al., 2023b), w.r.t. MASE and OWA.

4.3 FEW-SHOT FORECASTING

Setups. LLMs have recently demonstrated remarkable few-shot learning capabilities (Liu et al., 2023b). In this section, we assess whether our reprogrammed LLM retains this ability in forecasting tasks. We adhere to the setups in (Zhou et al., 2023a) for fair comparisons, and we evaluate on scenarios with limited training data (i.e., $\leq$ first 10% training time steps).

Results. Our brief 10% and 5% few-shot learning results are in Tab. 3 and Tab. 4 respectively. TIME-LLM remarkably excels over all baseline methods, and we attribute this to the successful knowledge activation in our reprogrammed LLM. Interestingly, both our approach and GPT4TS consistently surpass other competitive baselines, further underscoring the potential prowess of language models as proficient time series machines.

In the realm of 10% few-shot learning, our methodology realizes a **5%** MSE reduction in comparison to GPT4TS, without necessitating any fine-tuning on the LLM. In relation to recent SOTA models

Table 3: Few-shot learning on 10% training data. We use the same protocol in Tab. 1. All results are averaged from four different forecasting horizons: $H \in \{96, 192, 336, 720\}$. Our full results are in Appendix E.

Methods	TIME-LLM (Ours)		GPT4TS (2023a)		DLinear (2023)		PatchTST (2023)		TimesNet (2023)		FEDformer (2022)		Autoformer (2021)		Stationary (2022)		ETSformer (2022)		LightTS (2022a)		Informer (2021)		Reformer (2020)	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
<i>ETTh1</i>	0.556	0.522	0.590	0.525	0.691	0.600	0.633	0.542	0.869	0.628	0.639	0.561	0.702	0.596	0.915	0.639	1.180	0.834	1.375	0.877	1.199	0.809	1.249	0.833
<i>ETTh2</i>	0.370	0.394	0.397	0.421	0.605	0.538	0.415	0.431	0.479	0.465	0.466	0.475	0.488	0.499	0.462	0.455	0.894	0.713	2.655	1.160	3.872	1.513	3.485	1.486
<i>ETTm1</i>	0.404	0.427	0.464	0.441	0.411	0.429	0.501	0.466	0.677	0.537	0.722	0.605	0.802	0.628	0.797	0.578	0.980	0.714	0.971	0.705	1.192	0.821	1.426	0.856
<i>ETTm2</i>	0.277	0.323	0.293	0.335	0.316	0.368	0.296	0.343	0.320	0.353	0.463	0.488	1.342	0.930	0.332	0.366	0.447	0.487	0.987	0.756	3.370	1.440	3.978	1.587
<i>Weather</i>	0.234	0.273	0.238	0.275	0.241	0.283	0.242	0.279	0.279	0.301	0.284	0.324	0.300	0.342	0.318	0.323	0.318	0.360	0.289	0.322	0.597	0.495	0.546	0.469
<i>ECL</i>	0.175	0.270	0.176	0.269	0.180	0.280	0.180	0.273	0.323	0.392	0.346	0.427	0.431	0.478	0.444	0.480	0.660	0.617	0.441	0.489	1.195	0.891	0.965	0.768
<i>Traffic</i>	0.429	0.306	0.440	0.310	0.447	0.313	0.430	0.305	0.951	0.535	0.663	0.425	0.749	0.446	1.453	0.815	1.914	0.936	1.248	0.684	1.534	0.811	1.551	0.821
1 st Count	7		1		0		1		0		0		0		0		0		0		0		0	

Table 4: Few-shot learning on 5% training data. We use the same protocol in Tab. 1. All results are averaged from four different forecasting horizons: $H \in \{96, 192, 336, 720\}$. Our full results are in Appendix E.

Methods	TIME-LLM (Ours)		GPT4TS (2023a)		DLinear (2023)		PatchTST (2023)		TimesNet (2023)		FEDformer (2022)		Autoformer (2021)		Stationary (2022)		ETSformer (2022)		LightTS (2022a)		Informer (2021)		Reformer (2020)	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	0.627	0.543	0.681	<u>0.560</u>	0.750	0.611	0.694	0.569	0.925	0.647	<u>0.658</u>	0.562	0.722	0.598	0.943	0.646	1.189	0.839	1.451	0.903	1.225	0.817	1.241	0.835
ETTh2	0.382	0.418	<u>0.400</u>	<u>0.433</u>	0.694	0.577	0.827	0.615	0.439	0.448	0.463	0.454	0.441	0.457	0.470	0.489	0.809	0.681	3.206	1.268	3.922	1.653	3.527	1.472
ETTh1	<u>0.425</u>	<u>0.434</u>	0.472	0.450	0.400	0.417	0.526	0.476	0.717	0.561	0.730	0.592	0.796	0.620	0.857	0.598	1.125	0.782	1.123	0.765	1.163	0.791	1.264	0.826
ETTh2	0.274	0.323	<u>0.308</u>	<u>0.346</u>	0.399	0.426	0.314	0.352	0.344	0.372	0.381	0.404	0.388	0.433	0.341	0.372	0.534	0.547	1.415	0.871	3.658	1.489	3.581	1.487
Weather	0.260	0.309	<u>0.263</u>	0.301	0.263	0.308	0.269	<u>0.303</u>	0.298	0.318	0.309	0.353	0.310	0.353	0.327	0.328	0.333	0.371	0.305	0.345	0.584	0.527	0.447	0.453
ECL	<u>0.179</u>	0.268	0.178	<u>0.273</u>	0.176	0.275	0.181	0.277	0.402	0.453	0.266	0.353	0.346	0.404	0.627	0.603	0.800	0.685	0.878	0.725	1.281	0.929	1.289	0.904
Traffic	<u>0.423</u>	<u>0.298</u>	0.434	0.305	0.450	0.317	0.418	0.296	0.867	0.493	0.676	0.423	0.833	0.502	1.526	0.839	1.859	0.927	1.557	0.795	1.591	0.832	1.618	0.851
1 st Count	5		<u>2</u>		1		1		0		0		0		0		0		0		0		0	

such as PatchTST, DLinear, and TimesNet, our average enhancements surpass **8%**, **12%**, and **33%** w.r.t. MSE. Analogous trends are discernible in the 5% few-shot learning scenarios, where our average advancement over GPT4TS exceeds **5%**. When compared with PatchTST, DLinear, and TimesNet, TIME-LLM manifests a striking average improvement of over **20%**.

4.4 ZERO-SHOT FORECASTING

Setups. Beyond few-shot learning, LLMs hold potential as effective zero-shot reasoners (Kojima et al., 2022). In this section, we evaluate the zero-shot learning capabilities of the reprogrammed LLM within the framework of cross-domain adaptation. Specifically, we examine how well a model performs on a dataset ♣ when it is optimized on another dataset ♠, where the model has not encountered any data samples from the dataset ♣. Similar to few-shot learning, we use long-term forecasting protocol and evaluate on various cross-domain scenarios utilizing the ETT datasets.

Results. Our brief results are in Tab. 5. TIME-LLM consistently outperforms the most competitive baselines by a large margin, over **14.2%** w.r.t. the second-best in MSE reduction. Considering the few-shot results, we observe that reprogramming an LLM tends to yield significantly better results in data scarcity scenarios. For example, our overall error reductions w.r.t. GPT4TS in 10% few-shot forecasting, 5% few-shot forecasting, and zero-shot forecasting are increasing gradually: **7.7%**, **8.4%**, and **22%**. Even when benchmarked against LLMTIME, the most recent approach in this field, with the backbone LLM of comparable size (7B), TIME-LLM shows a substantial improvement exceeding **75%**. We attribute this to our approach being better at activating the LLM’s knowledge transfer and reasoning capabilities in a resource-efficient manner when performing time series tasks.

4.5 MODEL ANALYSIS

Language Model Variants. We compare two representative backbones with varying capacities (A.1-4 in Tab. 6). Our results indicate that the scaling law retain after the LLM reprogramming. We adopt Llama-7B by default in its full capacity, which manifestly outperforms its 1/4 capacity variant (A.2; inclusive of the first 8 Transformer layers) by **14.5%**. An average MSE reduction of **14.7%** is observed over GPT-2 (A.3), which slightly outperforms its variant GPT-2 (6) (A.4) by 2.7%.

Cross-modality Alignment. Our results in Tab. 6 indicate that ablating either patch reprogramming or Prompt-as-Prefix hurts knowledge transfer in reprogramming the LLM for effective time series forecasting. In the absence of representation alignment (B.1), we observe a notable average performance degradation of **9.2%**, which becomes more pronounced (exceeding **17%**) in few-shot tasks. In TIME-LLM, the act of prompting stands as a pivotal element in harnessing the LLM’s capacity for understanding the inputs and tasks. Ablation of this component (B.2) results in over **8%** and **19%** degradation in standard and few-shot forecasting tasks, respectively. We find that removing the input statistics (C.1) hurts the most, resulting in an average increase of **10.2%** MSE. This is an

Table 5: Zero-shot learning results. **Red**: the best, **Blue**: the second best. Appendix E shows our detailed results.

Methods	TIME-LLM (Ours)		GPT4TS (2023a)		LLMTime (2023)		DLinear (2023)		PatchTST (2023)		TimesNet (2023)	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1 → ETTh2	0.353	0.387	0.406	0.422	0.992	0.708	0.493	0.488	<u>0.380</u>	<u>0.405</u>	0.421	0.431
ETTh1 → ETTm2	0.273	0.340	0.325	0.363	1.867	0.869	0.415	0.452	<u>0.314</u>	<u>0.360</u>	0.327	0.361
ETTh2 → ETTh1	0.479	0.474	0.757	0.578	1.961	0.981	0.703	0.574	<u>0.565</u>	<u>0.513</u>	0.865	0.621
ETTh2 → ETTm2	0.272	0.341	0.335	0.370	1.867	0.869	0.328	0.386	<u>0.325</u>	<u>0.365</u>	0.342	0.376
ETTh1 → ETTh2	0.381	0.412	<u>0.433</u>	0.439	0.992	0.708	0.464	0.475	0.439	<u>0.438</u>	0.457	0.454
ETTh1 → ETTm2	0.268	0.320	0.313	0.348	1.867	0.869	0.335	0.389	<u>0.296</u>	<u>0.334</u>	0.322	0.354
ETTh2 → ETTh1	0.354	0.400	0.435	0.443	0.992	0.708	0.455	0.471	<u>0.409</u>	<u>0.425</u>	0.435	0.443
ETTh2 → ETTm1	0.414	0.438	0.769	0.567	1.933	0.984	0.649	0.537	<u>0.568</u>	<u>0.492</u>	0.769	0.567

Table 6: Ablations on ETTh1 and ETTm1 in predicting 96 and 192 steps ahead (MSE reported). **Red**: the best.

Variant	Long-term Forecasting				Few-shot Forecasting			
	ETTh1-96	ETTh1-192	ETThm1-96	ETThm1-192	ETTh1-96	ETTh1-192	ETThm1-96	ETThm1-192
A.1 Llama (Default; 32)	0.362	0.398	0.272	0.310	0.448	0.484	0.346	0.373
A.2 Llama (8)	0.389	0.412	0.297	0.329	0.567	0.632	0.451	0.490
A.3 GPT-2 (12)	0.385	0.419	0.306	0.332	0.548	0.617	0.447	0.509
A.4 GPT-2 (6)	0.394	0.427	0.311	0.342	0.571	0.640	0.468	0.512
B.1 w/o Patch Reprogramming	0.410	0.412	0.310	0.342	0.498	0.570	0.445	0.487
B.2 w/o Prompt-as-Prefix	0.398	0.423	0.298	0.339	0.521	0.617	0.432	0.481
C.1 w/o Dataset Context	0.402	0.417	0.298	0.331	0.491	0.538	0.392	0.447
C.2 w/o Task Instruction	0.388	0.420	0.285	0.327	0.476	0.529	0.387	0.439
C.3 w/o Statistical Context	0.391	0.419	0.279	0.347	0.483	0.547	0.421	0.461

Table 7: Efficiency analysis of TIME-LLM on ETTh1 in forecasting different steps ahead.

Length	ETTh1-96			ETTh1-192			ETTh1-336			ETTh1-512		
	Param. (M)	Mem. (MiB)	Speed(s/iter)	Param. (M)	Mem. (MiB)	Speed(s/iter)	Param. (M)	Mem. (MiB)	Speed(s/iter)	Param. (M)	Mem. (MiB)	Speed(s/iter)
D.1 Llama (32)	3404.53	32136	0.517	3404.57	33762	0.582	3404.62	37988	0.632	3404.69	39004	0.697
D.2 Llama (8)	975.83	11370	0.184	975.87	12392	0.192	975.92	13188	0.203	976.11	13616	0.217
D.3 w/o LLM	6.39	3678	0.046	6.42	3812	0.087	6.48	3960	0.093	6.55	4176	0.129

anticipated as external knowledge can be naturally incorporated via prompting to facilitate the learning and inference. Additionally, providing the LLM with clear task instructions and input context (e.g., dataset captioning) is also beneficial (i.e., **C.2** and **C.1**; eliciting over **7.7%** and **9.6%**, respectively).

Reprogramming Interpretation. We provide a case study on ETTh1 of reprogramming 48 time series patches with 100 text prototypes in Fig. 5. The top 4 subplots visualize the optimization of reprogramming space from randomly-initialized (**a**) to well-optimized (**d**). We find only a small set of prototypes (columns) participated in reprogramming the input patches (rows) in subplot (**e**). Also, patches undergo different representations through varying combinations of prototypes. This indicates: (1) text prototypes learn to summarize language cues, and a select few are highly relevant for representing information in local time series patches, which we visualize by randomly selecting 10 in subplot (**f**). Our results suggest a high relevance to the words that describe time series properties (i.e., word sets 1 and 2); (2) patches usually have different underlying semantics, necessitating different prototypes to represent.

Reprogramming Efficiency. Tab. 7 provides an overall efficiency analysis of TIME-LLM with and without the backbone LLM. Our proposed reprogramming network itself (**D.3**) is lightweight in activating the LLM’s ability for time series forecasting (i.e., fewer than 6.6 million *trainable* parameters; only around **0.2%** of the total parameters in Llama-7B), and the overall efficiency of TIME-LLM is actually capped by the leveraged backbones (e.g., **D.1** and **D.2**). This is favorable even compared to the parameter-efficient fine-tuning methods (e.g., QLoRA (Dettmers et al., 2023)) in balancing task performance and efficiency.

5 CONCLUSION AND FUTURE WORK

TIME-LLM shows promise in adapting frozen large language models for time series forecasting by reprogramming time series data into text prototypes more natural for LLMs and providing natural language guidance via Prompt-as-Prefix to augment reasoning. Evaluations demonstrate the adapted LLMs can outperform specialized expert models, indicating their potential as effective time series machines. Our results also provide a novel insight that time series forecasting can be cast as yet another “language” task that can be tackled by an off-the-shelf LLM to achieve state-of-the-art performance through our Time-LLM framework. Further research should explore optimal reprogramming representations, enrich LLMs with explicit time series knowledge through continued pre-training, and build towards multimodal models with joint reasoning across time series, natural language, and other modalities. Furthermore, applying the reprogramming framework to equip LLMs with broader time series analytical abilities or other new capabilities should also be considered.

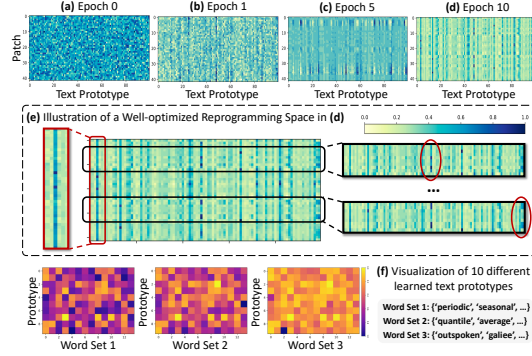


Figure 5: A showcase of patch reprogramming.